Health Check System

Generated during a Claude Code session - This is a work in progress and represents the rough first pass at ideation on this concept.

Overview

The Health Check system gives admins and service operations staff a real-time signal about how accounts are performing. It operates at two levels — portfolio (dashboard) and per-account — and surfaces as a Red / Yellow / Green tier backed by a numeric score (0–100).

Health is not a passive reporting layer. Incidents and sensitive events **immediately flip an account to Red** and simultaneously generate actionable tasks for the service operations team (managers, supervisors, operators). The score system handles the Yellow/Green gradations for accounts without active incidents.

Critically, **a resolved incident does not erase history**. The `HealthEvents` table is a permanent audit log. Score decay and resolution affect *current health state*, but the historical record drives a separate reporting and prevention layer that surfaces patterns over time — helping managers proactively address the conditions that lead to incidents before they recur.

This doc covers the data model, scoring architecture, event pipeline, task generation, new roles, UI surfaces, HubSpot integration, reporting and prevention, and the hook for the future ticketing system.

Core Concepts

Three Systems, One Event Source

Every meaningful occurrence — an incident ticket, a crew setting off an alarm, a missed service — is written as a `HealthEvent`. Three downstream systems read from that table independently:

1. **Health scoring** — computes and stores a tier + numeric score per account and deal (current state)
2. **Task generation** — creates operational tasks assigned to service staff based on rule templates (response)
3. **Reporting and prevention** — surfaces historical patterns to help prevent future events (trend intelligence)

```
Event occurs (incident, sensitive event, missed service, etc.)
 │
 ▼
HealthEvent written (permanent — never deleted)
 │
 ├──► Health scoring         [current state]
 │     ├── Incident / sensitive event → immediately RED (bypasses score math)
 │     └── Other events → score recalculated, tier derived from range
 │
 ├──► Task generation        [response]
 │     ├── today tasks     → Service Manager
 │     ├── this_week tasks → Supervisor
 │     └── this_month tasks→ Operator
 │
 └──► Reporting & prevention  [trend intelligence]
       ├── Event frequency reports (weekly / monthly / quarterly)
       ├── Risk profiles — pattern detection across resolved events
       └── Proactive flags — accounts trending toward repeat incidents
```

Key distinction: health score reflects *current state* (resolves when incidents close and decay clears). Reporting reflects *historical patterns* (never resets — resolved events are still counted). An account can be Green today but still appear on a risk watchlist because of event frequency in the past 12 months.

Score Tiers

Tier	Range	Meaning
Green	70–100	Healthy, no action needed
Yellow	40–69	Needs attention — watch closely
Red	0–39	At risk — OR immediate incident flag

Incident override: Any open `HealthEvent` with `category: 'incident' | 'sensitive_event'` forces the account to Red regardless of numeric score. The score is still computed and stored — it becomes relevant again once the event is resolved and decay has cleared.

Score Dimensions

Health is a weighted composite of five dimensions. Weights are configurable — a new account without feedback history should not be penalized for missing signals.

Dimension	What it measures	Primary signal source
`serviceQuality`	Incident rate, customer satisfaction, repeat offenses	HealthEvents (tickets, incidents)
`contractHealth`	Deal stage, renewal proximity, deal value trend	HubSpot deals via CRM adapter
`activityRecency`	Last service date, last deal activity	HubSpot deal activity
`customerEngagement`	Contact frequency, responsiveness	HubSpot contacts and activity
`issueResolution`	Open vs. resolved events, task completion rate	HealthEvents + ServiceTasks

Score Decay

`HealthEvent` records carry a `decayDays` field. A high-severity alarm trigger with `scoreImpact: -20` and `decayDays: 30` loses its full negative impact linearly to `0` over 30 days. This prevents stale incidents from permanently suppressing an account's score after the situation has been handled.

Task completion feeds back positively: completing a remediation task adds a `+N` credit to `issueResolution`, making the feedback loop visible.

Data Model

`AccountHealth` (DynamoDB)

```
PK: accountId

SK: "CURRENT"                    ← overwritten on each compute

  "SNAPSHOT#<ISO timestamp>"    ← retained for trend history (keep last 12)

score: number                    ← 0–100

tier: 'green' | 'yellow' | 'red'

incidentOverride: boolean        ← true if tier is forced Red by open incident

components: {

 serviceQuality:     { score: number, tier: string, factors: string[] }

 contractHealth:     { score: number, tier: string, factors: string[] }

 activityRecency:    { score: number, tier: string, factors: string[] }

 customerEngagement: { score: number, tier: string, factors: string[] }

 issueResolution:    { score: number, tier: string, factors: string[] }

}

lastComputedAt: string           ← ISO timestamp
```

computedBy: 'system' | 'webhook' | 'scheduled' | 'manual'

`DealHealth` (DynamoDB)

PK: dealId

score: number

tier: 'green' | 'yellow' | 'red'

incidentOverride: boolean

components: {

 contractHealth: { score: number, tier: string, factors: string[] }

 activityRecency: { score: number, tier: string, factors: string[] }

 issueResolution: { score: number, tier: string, factors: string[] }

}

lastComputedAt: string

Deal health rolls up into account health — an account with multiple deals uses a weighted aggregate (active deals weighted more heavily than draft).

`HealthEvents` (DynamoDB)

The root table. Owned here; the future ticketing system publishes into it.

PK: accountId

SK: "<ISO timestamp>#<eventId>" ← ULID suffix for uniqueness; sortable by time

eventId: string

dealId?: string

category: 'incident' | 'sensitive_event' | 'customer_complaint' | 'missed_service'

| 'payment_issue' | 'quality_flag' | 'health_decline'

severity: 'low' | 'medium' | 'high' | 'critical'

description: string

reportedAt: string ← ISO

resolvedAt?: string ← set when issue is closed; stops incident override

source: 'ticket' | 'manual' | 'webhook' | 'system'

scoreImpact: number ← negative integer (e.g. -20); 0 once decayed

decayDays: number ← impact reaches 0 after N days

repeatCount?: number ← system-tracked; same category+accountId within 180 days

GSI: dealId (to fetch all events for a deal)

GSI: resolvedAt (to query open events efficiently)

GSI: category+reportedAt (to support reporting queries by event type over time)

Data retention: `HealthEvents` records are **never deleted**. `resolvedAt` and score decay affect the *current health state* computation only. The full event history is the foundation for the reporting and prevention layer — purging it would eliminate the ability to detect patterns and generate trend reports.

Repeat offense escalation: If `repeatCount` for `category: 'incident' | 'sensitive_event'` on the same account reaches 3 within 180 days, severity is automatically escalated one level and a `contract_review` task is generated regardless of the original event's rule template.

`ServiceTasks` (DynamoDB)

PK: assigneeId ← userId or role pool ID for unassigned tasks

SK: "<dueDate>#<taskId>" ← enables range queries by due date

taskId: string

sourceEventId: string

accountId: string

dealId?: string

```
title: string

category: 'incident_report' | 'follow_up_call' | 'remediation_visit'

        | 'account_review' | 'crew_debrief' | 'contract_review' | 'spot_check'

priority: 'critical' | 'high' | 'medium' | 'low'

dueWindow: 'today' | 'this_week' | 'this_month'

dueDate: string                ← ISO; computed from dueWindow at creation time

status: 'pending' | 'in_progress' | 'completed' | 'dismissed'

assigneeRole: 'service_manager' | 'supervisor' | 'operator'

createdAt: string

completedAt?: string

dismissedReason?: string

GSI: accountId        ← fetch all open tasks for an account

GSI: dueDate          ← fetch all tasks due today across all assignees

GSI: status+dueDate   ← overdue task detection
```

`TaskRules` (DynamoDB)

Configurable rule templates — task generation is a data change, not a code deploy.

```
PK: category                   ← HealthEvent category

SK: severity

taskTemplates: Array<{

 title: string

 category: string              ← ServiceTask category

 assigneeRole: 'service_manager' | 'supervisor' | 'operator'
```

```
 dueWindow: 'today' | 'this_week' | 'this_month'

 priority: 'critical' | 'high' | 'medium' | 'low'

}>
```

Example rule — `PK: 'sensitive_event', SK: 'high'`:

```
{

 "taskTemplates": [

   { "title": "File incident report", "category": "incident_report", "assigneeRole":
"service_manager", "dueWindow": "today", "priority": "critical" },

   { "title": "Call account contact", "category": "follow_up_call", "assigneeRole":
"service_manager", "dueWindow": "today", "priority": "critical" },

   { "title": "Schedule crew debrief", "category": "crew_debrief", "assigneeRole": "supervisor",
"dueWindow": "this_week", "priority": "high" }

 ]

}
```

Event Pipeline

Immediate Red Trigger

`incident` and `sensitive_event` categories short-circuit the score math:

```
writeHealthEvent(accountId, { category: 'incident', severity: 'high', ... })

 → setIncidentOverride(accountId, true)          ← account is Red immediately

 → generateTasks(eventId, rules)                 ← tasks created from TaskRules

 → writeAccountHealth(accountId, { tier: 'red', incidentOverride: true, ... })

 → writeBackToHubSpot(accountId, 'red')          ← CRM property updated
```

Override clears when `resolvedAt` is set on the `HealthEvent` and no other open incident events remain for that account.

Score-Driven Recalculation

For non-incident events (quality flags, activity recency decay, deal stage changes):

computeAccountHealth(accountId):

1. Fetch active deals → computeDealHealth for each
2. Fetch open HealthEvents (last 90 days) → apply scoreImpact + decay
3. Fetch CRM signals (deal stage, activity recency, contact frequency)
4. Check for open incident override → if yes, tier = 'red' regardless
5. Compute weighted composite score from five dimensions
6. Write CURRENT to AccountHealth
7. Write SNAPSHOT#<timestamp> for trend history
8. Write-back tier + score to HubSpot company properties

Recompute Triggers

Trigger	**Notes**
HubSpot deal stage webhook	Existing webhook handler extended to call `computeAccountHealth`
Ticket created / resolved (future)	Ticketing system publishes HealthEvent → recompute
Task completed	`issueResolution` sub-score updated
Manual admin action	UI button on account detail
Nightly scheduled job	Applies score decay even when no events occur

---

Task Generation

Tasks are generated immediately when a `HealthEvent` is written. The engine:

1. Reads `TaskRules[category][severity]` from DynamoDB
2. Creates one `ServiceTask` per template, setting `dueDate` from `dueWindow`
3. Assigns to role pool if no specific assignee is known yet (manager assigns later)
4. If `repeatCount >= 3`, adds a `contract_review` task regardless of rule template

Task Completion → Health Feedback

- **Completed** → positive `issueResolution` signal; score recalculated
- **Overdue** (past `dueDate`, still `pending`) → additional negative signal on next nightly compute
- **Dismissed with reason** → neutral; logged for audit; no score change

---

Role Expansion

The existing roles (admin, sales-manager, sales-rep) are the commercial side. Health Check introduces the operations side:

Role	Primary surface	Task scope	Notes
`service_manager`	Operational dashboard — account health + open tasks	Tasks for their assigned accounts	Assigns supervisor/operator tasks
`supervisor`	Task list + crew view	This week's field tasks	May manage multiple crews
`operator`	Daily task list	Today's tasks only	Field staff; mobile-first

Key open question: Is a service manager assigned to specific accounts (1:N relationship in DynamoDB) or do they own a region or crew? This determines how task routing and account-level filtering work. Resolve before Phase 2 implementation.

Permissions to add:

- `health:read` — view health scores and events (admin, service_manager, supervisor)

- `health:write` — create manual health events, trigger recompute (admin, service_manager)
- `tasks:read` — view task list (all ops roles)
- `tasks:write` — complete, dismiss, reassign tasks (all ops roles, scoped to their queue)

UI Surfaces

1. Admin Dashboard — Portfolio Health Scorecard

New section between the existing stat cards and the deal cards:

- Three stat cards: `X Healthy`, `Y Watch`, `Z At Risk` with RYG color treatment
- "Accounts Needing Attention" list — top 5 Red/Yellow accounts, sorted by score ascending; each row shows account name, primary driving factor, and a link to the account detail
- Trend indicator: "2 accounts declined since last week" (derived from SNAPSHOT history)

2. Service Manager Dashboard (new route)

`/dashboard/operations` — task-oriented primary view for service operations staff:

- Summary: tasks due today, overdue, completed this week
- Task list grouped by `dueWindow` (Today / This Week / This Month)
- Filter by account, priority, category
- Quick link to source account from each task

3. Operator / Supervisor Task List

`/tasks` — simplified task list, mobile-first:

- Today's tasks by default
- Tap to mark complete, add notes, flag as blocked
- No health score visibility — purely action-oriented

4. Account List — Health Column

New column in the accounts table with a colored `HealthBadge` (RYG dot + label). Filterable by tier. Consistent with the existing `StatusBadge` component pattern — likely a new `variant` or a separate `HealthBadge` primitive.

5. Account Detail — Health Tab

New tab alongside Deals and Site Details:

- Overall score with tier label and `incidentOverride` indicator if active
- Component breakdown: five rows (dimension, score bar, tier, contributing factors)
- Recent health events log (last 10, with severity chip, date, description, resolution status)
- Deal-level health summary for each active deal
- Manual recompute button (admin/service_manager only)

6. Deal Detail — Health Indicator

Lightweight: health tier badge in the deal detail header. Tooltip/popover shows the 2–3 key contributing factors. No full tab at the deal level — the account Health tab covers the rollup.

HubSpot Integration

Two HubSpot company properties to create as custom properties:

Property name	Type	Description
`health_tier`	string	`'green'` / `'yellow'` / `'red'`
`health_score`	number	0–100

Write-back follows the same pattern as site aggregates (`totalSqFootage`, `rooms` etc.) — computed in DynamoDB, then PATCHed to HubSpot via the CRM adapter. Two new methods on `CrmRepository`:

```
updateAccountHealth(accountId: string, tier: HealthTier, score: number): Promise<void>;

// Optional — reads HubSpot activity signals for scoring inputs:

getAccountActivitySignals(accountId: string): Promise<AccountActivitySignals>;
```

This gives HubSpot users CRM-native health visibility and enables HubSpot-side filters and workflow automations.

Ticketing System Integration (future)

The `HealthEvents` table is designed so the ticketing system plugs in without health scoring knowing about ticket internals.

Contract:

Ticket created → ticketing system writes HealthEvent (source: 'ticket')

→ health recompute triggered automatically

Ticket resolved → ticketing system sets resolvedAt on HealthEvent

→ if no other open incidents, incidentOverride clears

→ health recompute triggered

The ticketing system owns its own data store (tickets, comments, attachments). It publishes *only* the health-relevant signal to `HealthEvents`. Health scoring remains fully decoupled from ticket implementation details.

Sensitive event reporting: Field operators or supervisors can create a `HealthEvent` manually (via the task/ops UI) without a formal ticket — useful for events like an alarm trigger that need immediate health impact before a ticket is formally filed.

Reporting and Prevention

Health scores tell you *where things stand today*. Reports tell you *why things keep happening* and *what to do before the next incident*.

What Gets Reported

Account-level event history

- Full chronological log of all events (open and resolved) for the account's lifetime
- Visible on the account Health tab — not just the last 10, but paginated full history
- Filterable by category, severity, resolution status, date range

Portfolio-level trend reports (admin and service manager)

Report	Cadence	Content
Weekly digest	Weekly	New incidents this week, resolved vs. open, accounts that changed tier
Monthly summary	Monthly	Event counts by category, top 5 accounts by event frequency, task completion rate
Quarterly review	Quarterly	Year-over-year event trends, accounts on risk watchlist, crew/operator correlation if available

Event category breakdowns — which types of events are increasing or decreasing over time across the portfolio. A spike in `alarm_trigger` events across multiple accounts in the same month is a signal worth surfacing to management.

Risk Profiles and Watchlist

An account's **risk profile** is distinct from its health score:

- **Health score** = current state, resets as incidents resolve and decay clears
- **Risk profile** = rolling 12-month event frequency, never resets

An account can be Green (healthy current state) but flagged on the risk watchlist because it has had 4 resolved incidents in the past year. The watchlist surfaces these accounts so service managers can proactively schedule reviews, retrain crews, or revisit service agreements before another event occurs.

AccountHealth record additions for risk profile:

```
riskProfile: {

  eventCount12m: number        ← total HealthEvents in rolling 12 months

  incidentCount12m: number     ← incident + sensitive_event specifically

  lastEventAt: string          ← most recent event regardless of resolution

  watchlist: boolean           ← true if incidentCount12m >= threshold
```

```
  watchlistReason?: string      ← e.g. "3 incidents in 12 months"

}
```

The watchlist threshold is configurable (default: 2+ incidents in 12 months). Watchlist status does not affect the health score tier — it is a separate signal displayed alongside the score.

Proactive Flags

The nightly compute job evaluates patterns beyond the current score:

- Account has had the same event category 2+ times in 90 days (even if all resolved) → generate a `account_review` task for service manager flagged as "recurring pattern"
- Account health score has been Yellow for 30+ consecutive days without an incident (gradual decline, not event-driven) → flag for review
- Account has no recorded events for 180+ days but score has been declining (signals missing data — possibly events not being reported)

These flags generate tasks in `ServiceTasks` with `priority: 'medium'` and `dueWindow: 'this_month'` so they surface in the manager's task list without urgency pressure.

Report Delivery

Reports are not just in-app views — they should also be deliverable outside the portal:

- **In-app:** report views on the `/dashboard/operations` route for service managers; portfolio summary on admin dashboard
- **Email digest:** scheduled reports sent to admins and service managers (requires a notification/email service — not yet scoped)
- **Export:** CSV/PDF export of event history for an account or the full portfolio (useful for contract reviews and client meetings)

Report generation should be triggered by the nightly scheduled job. Generated reports can be stored as structured records in a `Reports` DynamoDB table or as files in S3, depending on size and format needs — defer the decision until email/export is scoped.

Implementation Phases

Phase	Scope	Depends on
1 — Foundation	`AccountHealth`, `DealHealth`, `HealthEvents`, `ServiceTasks`, `TaskRules` DynamoDB tables; `computeAccountHealth` service; basic scoring from deal stage signals only; incident override logic; data retention enforced (no deletes on `HealthEvents`)	Existing DynamoDB + CRM adapter
2 — Dashboard	Portfolio health stat cards + "Accounts Needing Attention" in admin dashboard; health column on accounts list	Phase 1
3 — Account surfaces	Health tab on account detail (score, component breakdown, paginated full event history); `HealthBadge` primitive; health badge in account header	Phase 1
4 — HubSpot write-back	`updateAccountHealth` on `CrmRepository`; custom HubSpot property creation; webhook trigger extended	Phase 1
5 — Task system	`TaskRules` seeding; task generation engine; `ServiceTasks` writes; `/dashboard/operations` and `/tasks` routes; ops role permissions	Phase 1, role expansion decisions
6 — Deal health	Deal-level scoring; health badge on deal detail; deal rollup into account score	Phase 1

Phase	Scope	Depends on
7 — Reporting and prevention	Risk profiles + watchlist in `AccountHealth`; nightly proactive flag generation; weekly/monthly report views in `/dashboard/operations`; portfolio trend report on admin dashboard	Phase 1, Phase 5 (nightly job)
8 — Report delivery	CSV/PDF export of event history per account and portfolio-wide; email digest (requires notification/email service — not yet scoped)	Phase 7, email service decision
9 — Ticketing bridge	`HealthEvents` write path from ticketing system; event log in Health tab pulls from both manual and ticket sources	Ticketing system scoped

---

Open Questions

#	Question	Why it matters
1	How are service managers assigned to accounts — by account relationship, region, or crew?	Determines task routing and filtering architecture
2	Can admins or managers manually create `HealthEvents` outside of tickets or webhooks?	Needed for sensitive events before a ticket is formally filed
3	Does task completion automatically improve the health score, or does a	Affects `issueResolution` sub-score trigger

#	Question	Why it matters
	manager need to confirm the issue is resolved?	
4	Is the operator task interface mobile web or a future native app?	Sets investment level for responsive task UI now
5	What are the exact SLA windows — if a `today` task is not completed by EOD, who is notified and how?	Required before nightly overdue detection is meaningful
6	Can a sales rep see their account's health score and open tasks, or is operations data walled off from the commercial side?	Permission scoping decision
7	Should new accounts start at Green (decay over time) or neutral (no tier until enough signals exist)?	Baseline / onboarding grace period logic
8	Can an admin manually override a Red tier to Green (e.g., "we know about this, it's handled")? If so, does the override expire?	Manual override feature scope
9	What defines the exact list of `sensitive_event` subcategories (alarm trigger, security breach, etc.)?	`TaskRules` seeding and scoreImpact calibration
10	Score dimension weights — are they fixed globally, or configurable per account type (new vs. tenured)?	Scoring engine design